

SISTEMAS OPERATIVOS

SEMANA 08 – SESIÓN 01

Unidad de aprendizaje 2:
Gestión de memoria y gestión de archivos



Universidad
Tecnológica
del Perú

Logro de aprendizaje

Al finalizar la sesión el estudiante logra:

- Administrar la memoria y los archivos que componen un sistema operativo.



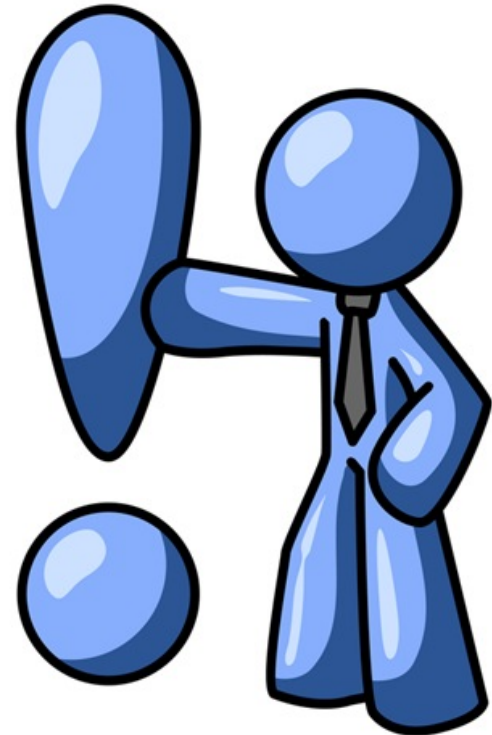
TEMARIO

- Gestión de Memoria.
- Administración básica de memoria.
- Asignación de memoria contigua.



Conocimientos previos

- Algoritmos



UTILIDAD

- Ejecución de programas cuyo requerimientos de memoria exceden a la memoria física.





DIRECCIONES DE MEMORIA

¿Qué son las Direcciones de Memoria?

Una dirección de memoria es un identificador único que especifica la ubicación de un byte específico en la memoria del sistema.



DIRECCIONES DE MEMORIA

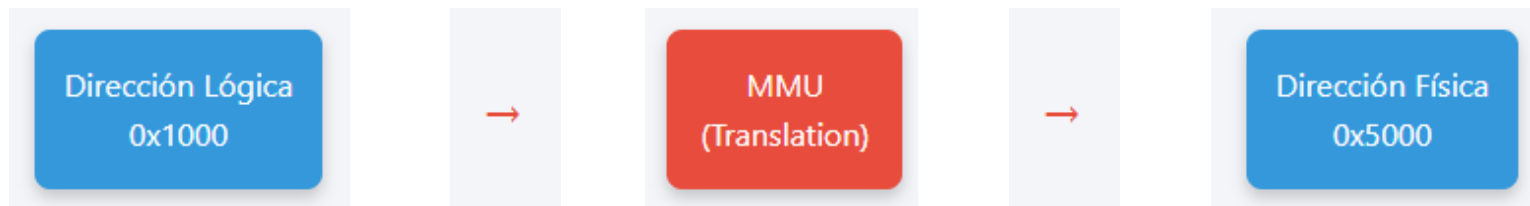


Tipos de Direcciones

- **Direcciones Lógicas (Virtuales):** Generadas por el procesador
- **Direcciones Físicas:** Ubicación real en la RAM
- **Direcciones Relativas:** Relativas a un punto base



DIRECCIONES DE MEMORIA



La MMU (Memory Management Unit) se encarga de traducir direcciones lógicas a físicas



DIRECCIONES DE MEMORIA

La memoria RAM puede ser vista como un conjunto finito de celdas que almacenan datos e instrucciones.

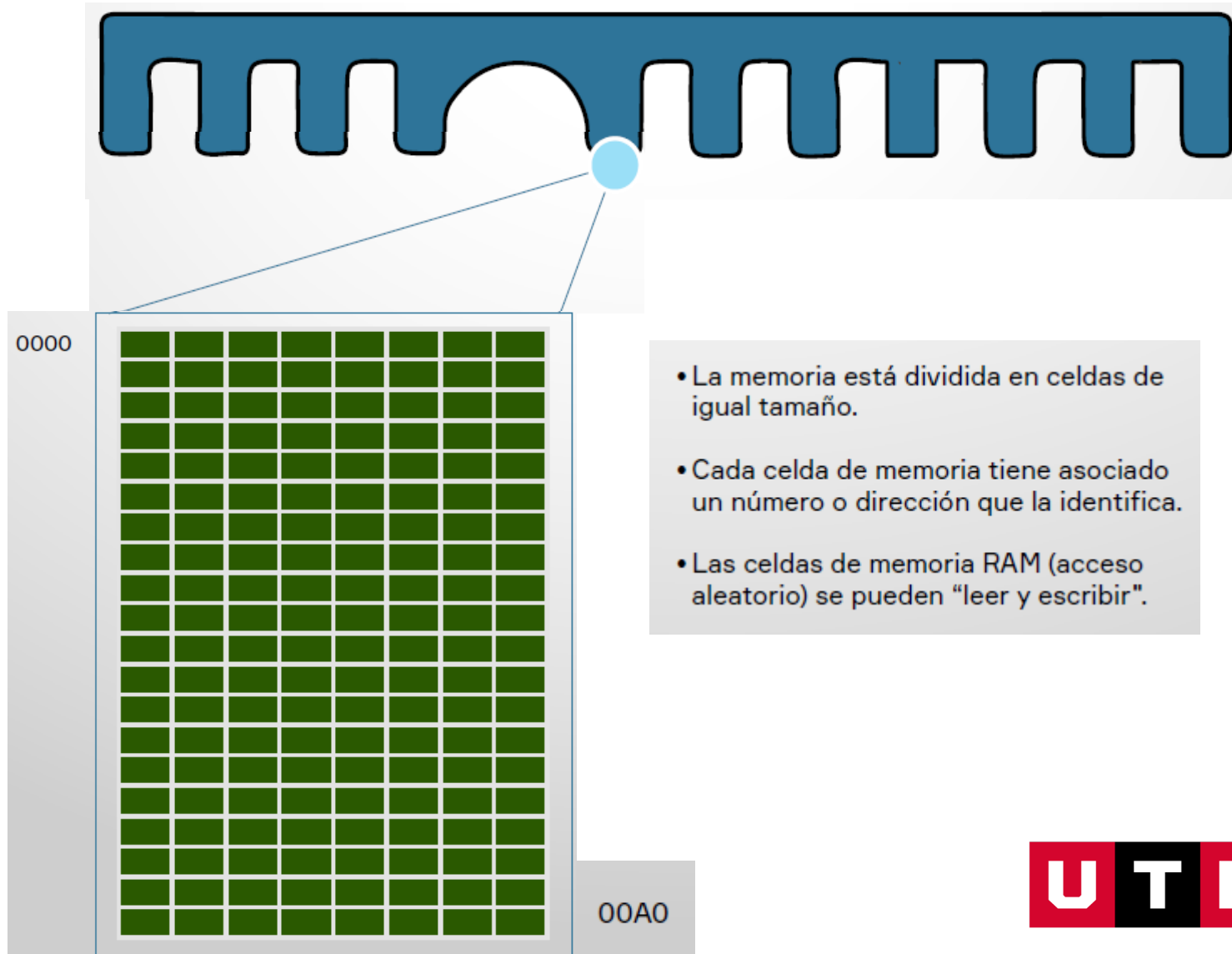
Cada celda tiene una dirección de memoria la cual se identifica con un número único expresado en código hexadecimal.

El sistema operativo asigna uno o varios rangos de direcciones de memoria a los procesos y lo hace de modo tal que estos rangos no se traslapen.

Los programas utilizan direcciones de memoria lógicas. Estas deben convertirse a direcciones de memoria físicas. Para agilizar este proceso, se cuenta con un elemento hardware llamado MMU (Memory Management Unit), que es el encargado de hacer esa conversión.



DIRECCIONES DE MEMORIA





MEMORIA VIRTUAL

¿Qué es la Memoria Virtual?

Técnica que permite que los programas usen más memoria de la que físicamente está disponible, creando la ilusión de un espacio de memoria más grande.



MEMORIA VIRTUAL

¿Qué permite?

Permite ejecutar procesos parcialmente cargados en memoria principal, mientras el resto está almacenado en disco, los cuales son referenciados fácilmente.



MEMORIA VIRTUAL

¿Cómo funciona?

Se utiliza el disco como almacén secundario de procesos, liberando al programador de la preocupación de que sus programas quepan o no en la memoria.



MEMORIA VIRTUAL

¿Cómo funciona?

Código que maneja condiciones de error poco común y casi nunca se ejecuta.

En muchos casos, se reserva más memoria de la necesaria (vectores, tablas, etc.)

Opciones y funciones del programa que se usan con muy poca frecuencia.



MEMORIA VIRTUAL

¿Cómo funciona?

El sistema operativo selecciona automáticamente qué fragmentos del proceso residen en memoria principal.

Cuando el programa hace referencia a una parte de su espacio de direcciones que no está en la memoria física, el sistema operativo recibe una alerta para buscar la parte faltante y volver a ejecutar la instrucción que falló.



MEMORIA VIRTUAL

¿Cómo funciona?

Ventajas

Antes se consideraba solo la posibilidad de mantener la totalidad del proceso en memoria para ejecutarlo.

En los casos donde se necesita todo el programa, es probable que no se requiera todo al mismo tiempo.

Como cada programa de usuario ocuparía menos memoria física, podrían ejecutarse más programas al mismo tiempo, aumentando la utilización de la CPU y la productividad, pero sin incrementar el tiempo de respuesta.

Se requeriría menos E/S para cargar o intercambiar cada uno de los programas de usuario, por lo que se ejecutarían más rápido.



MEMORIA VIRTUAL



Ventajas Principales

- 1 **Abstracción:** Los programas no necesitan saber la ubicación física real
- 2 **Protección:** Cada proceso tiene su propio espacio virtual
- 3 **Multiprogramación:** Múltiples programas pueden ejecutarse simultáneamente
- 4 **Intercambio:** Páginas pueden moverse entre RAM y disco



MEMORIA VIRTUAL

Ejemplo Práctico:

Si tienes 4GB de RAM pero ejecutas programas que en total requieren 6GB, la memoria virtual permite que funcionen usando el disco como extensión de la RAM.

Proceso A
2GB Virtual

Proceso B
3GB Virtual

Proceso C
1GB Virtual



RAM Física
4GB Total

PAGINACIÓN

Esquema de gestión de memoria que divide la memoria lógica y física en bloques de tamaño fijo llamados páginas y marcos de página respectivamente.



Componentes clave

- **Páginas:** Bloques de memoria lógica (ej: 4KB)
- **Marcos:** Bloques de memoria física del mismo tamaño
- **Tabla de Páginas:** Mapea páginas lógicas a marcos físicos
- **TLB:** Translation Lookaside Buffer (caché de traducción)



PAGINACIÓN

$\text{Dirección Física} = (\text{Número de Marco} \times \text{Tamaño de Página}) + \text{Desplazamiento}$

Página 0

Página 1

Página 2

Memoria Lógica



Tabla de Páginas

Traducción



Marco 3

Marco 1

Marco 7

Memoria Física



Tipos de Paginación

1

Paginación Simple

Características:

- Una sola tabla de páginas por proceso
- Traducción directa de dirección lógica a física
- Suitable para espacios de direcciones pequeños

2

Paginación Multinivel

Características:

- Múltiples niveles de tablas de páginas
- Reduce el espacio requerido para tablas grandes
- Ejemplo: Paginación de dos niveles, tres niveles

3

Tabla de Páginas Invertida

Características:

- Una entrada por marco físico, no por página lógica
- Ahorra espacio significativo
- Usado en sistemas como PowerPC, IA-64

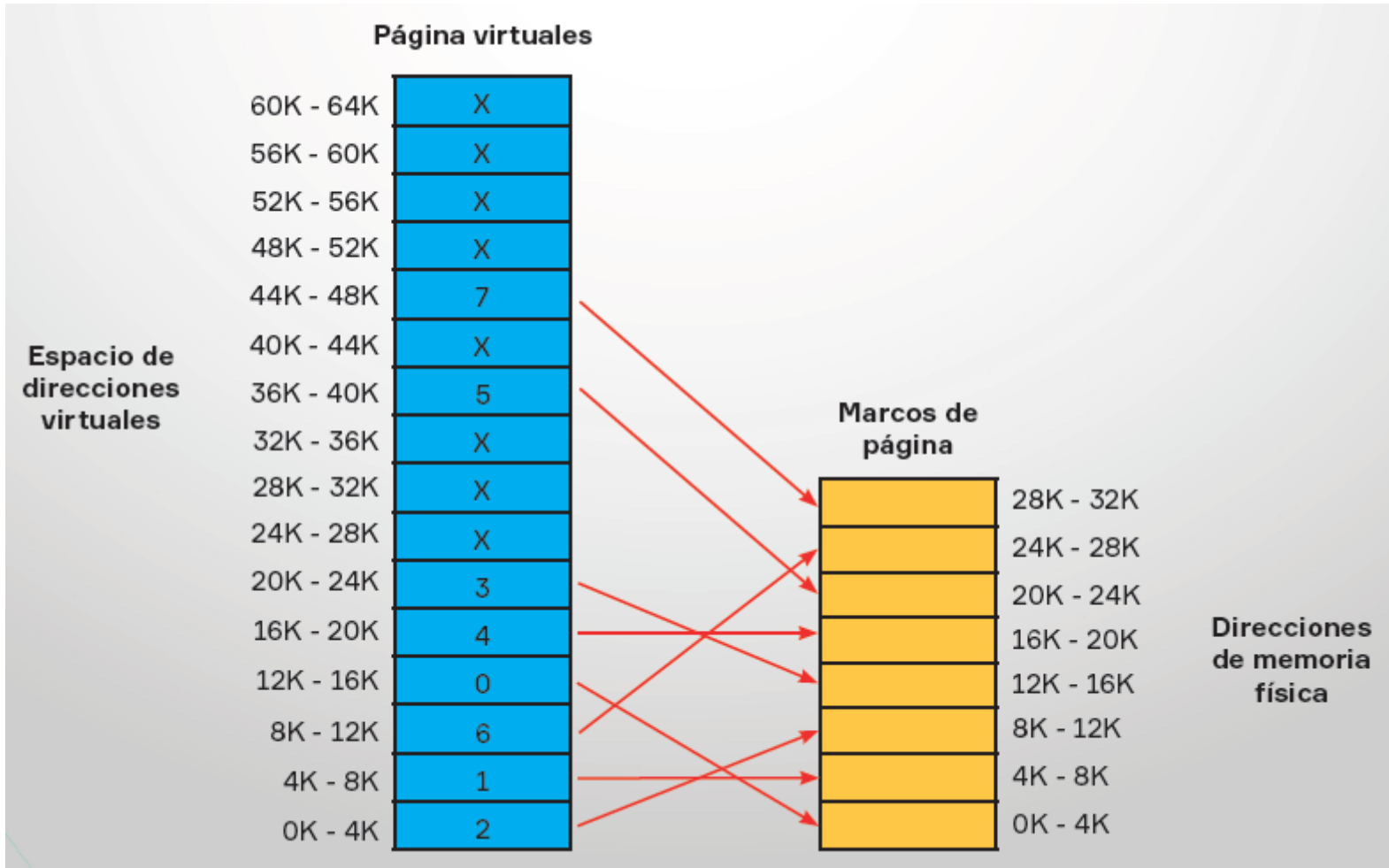


Tipos de Paginación

Tipo	Ventajas	Desventajas	Uso Típico
Simple	Implementación sencilla	Tabla muy grande	Sistemas pequeños
Multinivel	Ahorra espacio	Múltiples accesos	Sistemas modernos
Invertida	Muy eficiente en espacio	Búsqueda compleja	Sistemas especiales



Tipos de Paginación



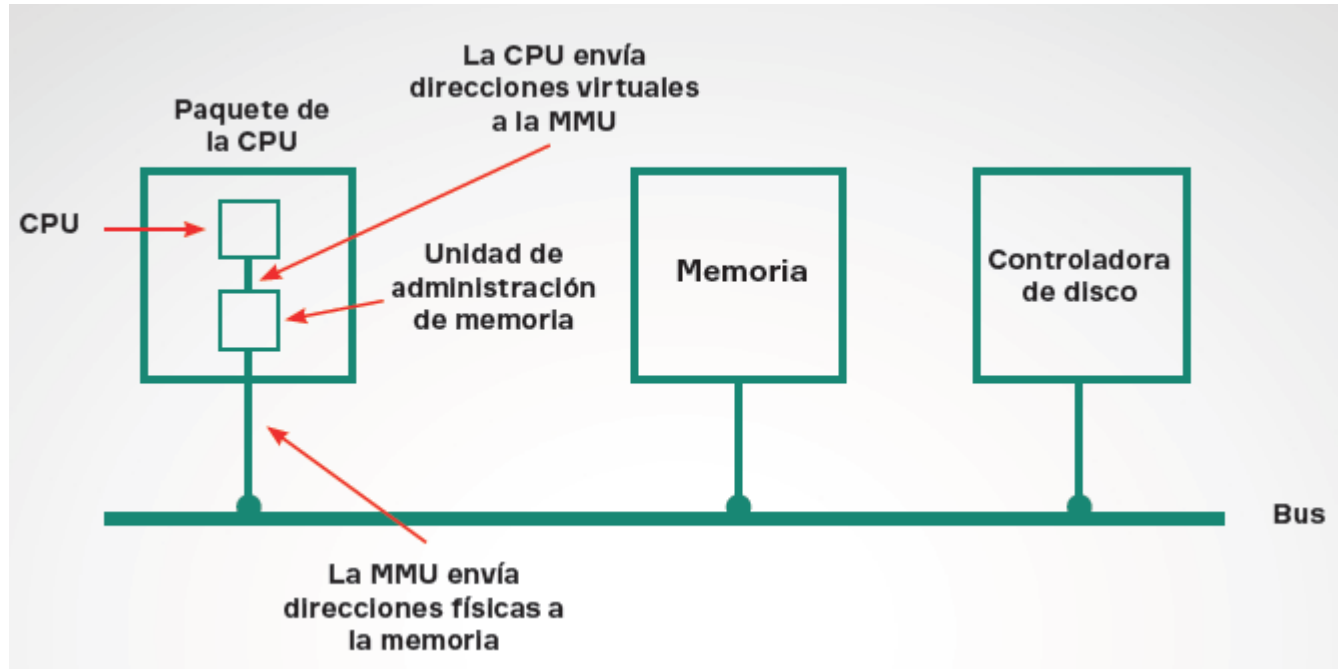


Tipos de Paginación

- Las direcciones generadas por el programa se conocen como direcciones virtuales y forman el espacio de direcciones virtuales.
- Cuando se utiliza memoria virtual, las direcciones virtuales no van directamente al bus de memoria. En vez de ello, van a la MMU para que este calcule su correspondiente dirección física.



Tipos de Paginación



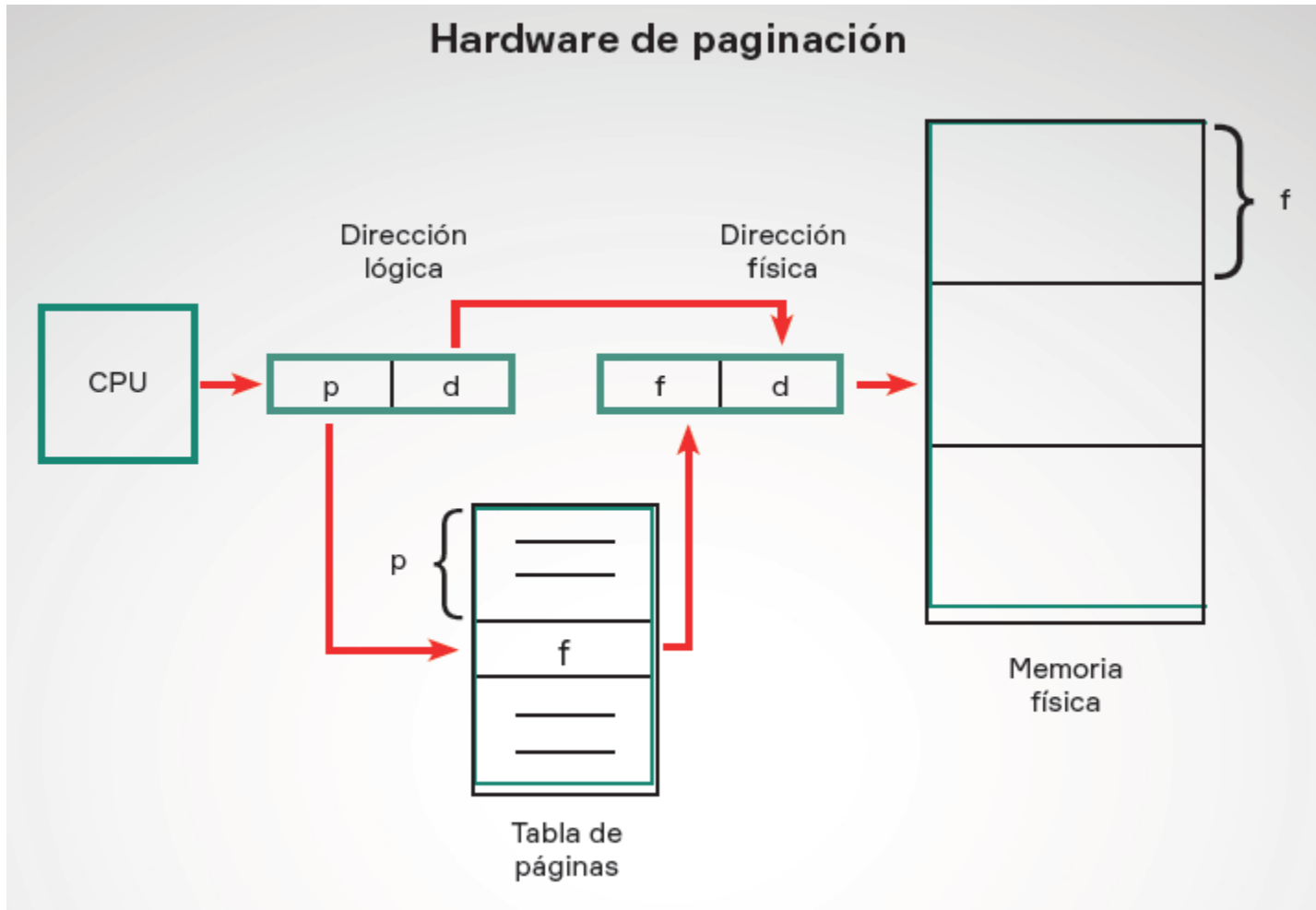


Tipos de Paginación

Número de página	Desplazamiento
p	d
$m - n$	n



Tipos de Paginación





Tipos de Paginación

Paginación: Ejemplo 1

page 0
page 1
page 2
page 3

Memoria
lógica

0	1
1	4
2	3
3	7

Tabla de
páginas

Número de
marco

0	
1	page 0
2	
3	page 2
0	page 1
1	
2	
3	page 3

Memoria
física



ALGORITMOS DE REEMPLAZO

¿Por qué Necesitamos Algoritmos de Reemplazo?

Cuando la memoria física se llena y llega una nueva página, ¿cuál página existente debemos reemplazar para minimizar los fallos de página futuros?



ALGORITMOS DE REEMPLAZO

Objetivos de los Algoritmos

- **Minimizar fallos de página:** Reducir accesos a disco
- **Maximizar el rendimiento:** Mejorar tiempo de respuesta
- **Eficiencia computacional:** Bajo overhead del algoritmo
- **Justicia:** No penalizar procesos específicos



ALGORITMOS DE REEMPLAZO

Ejemplo de Fallo de Página:

Si un proceso solicita la página 7, pero solo hay marcos para 4 páginas y todos están ocupados, el SO debe decidir cuál página reemplazar.

 El algoritmo óptimo minimizaría los fallos de página, pero requiere conocimiento del futuro



Métricas de Evaluación

$$\text{Tasa de Fallos} = (\text{Número de Fallos de Página} / \text{Total de Referencias}) \times 100\%$$

$$\text{Tiempo Efectivo de Acceso} = (1 - p) \times \text{Tiempo_RAM} + p \times \text{Tiempo_Disco}$$



ALGORITMOS FIFO Y LRU

Algoritmo FIFO (First In, First Out)

Principio: La página que ha estado más tiempo en memoria es la primera en ser reemplazada.

Implementación: Cola simple que mantiene el orden de llegada.

Ventajas: Muy simple de implementar y entender.

Desventajas: Puede sufrir de la Anomalía de Belady.

Ejemplo FIFO: Secuencia: 7, 0, 1, 2, 0, 3, 0, 4 (3 marcos)

Paso 1:

7	-	-
---	---	---

 Fallo

Paso 2:

7	0	-
---	---	---

 Fallo

Paso 3:

7	0	1
---	---	---

 Fallo



ALGORITMOS FIFO Y LRU

LRU (Least Recently Used)

Principio: Reemplaza la página que no ha sido usada por más tiempo.

Implementación: Pila o contador de tiempo para cada página.

Ventajas: Buen rendimiento, aproxima el comportamiento óptimo.

Desventajas: Mayor overhead computacional.



ALGORITMOS CLOCK Y ÓPTIMO

Algoritmo Clock (Segunda Oportunidad)

Principio: Aproximación a LRU usando un bit de referencia.

Funcionamiento:

1. Mantiene páginas en lista circular
2. Cada página tiene un bit de referencia
3. Si bit = 1: cambiar a 0 y continuar
4. Si bit = 0: seleccionar para reemplazo

Ventajas: Eficiente y buen rendimiento.

Página A
bit = 1



Página B
bit = 0



Página C
bit = 1



Puntero Clock



ALGORITMOS CLOCK Y ÓPTIMO

Algoritmo Óptimo (OPT/MIN)

Principio: Reemplaza la página que será referenciada más tarde en el futuro.

Características:

Teóricamente óptimo (mínimos fallos de página)

Imposible de implementar en la práctica

Usado como referencia para comparar otros algoritmos

Utilidad: Benchmark para evaluar algoritmos prácticos.



ALGORITMOS CLOCK Y ÓPTIMO

Algoritmo Óptimo (OPT/MIN)

Algoritmo	Complejidad	Rendimiento	Implementación
FIFO	$O(1)$	Regular	Muy simple
LRU	$O(1) - O(n)$	Bueno	Compleja
Clock	$O(1)$	Bueno	Simple
Óptimo	-	Perfecto	Imposible



PLANIFICACIÓN DE MEMORIA

¿Qué es la Planificación de Memoria?

Conjunto de algoritmos que determinan cómo asignar la memoria disponible a los procesos que la solicitan, optimizando el uso y minimizando la fragmentación.



PLANIFICACIÓN DE MEMORIA



Objetivos Principales

- **Minimizar fragmentación:** Reducir espacio desperdiciado
- **Maximizar utilización:** Usar eficientemente la memoria
- **Tiempo de respuesta:** Asignación rápida de memoria
- **Justicia:** Acceso equitativo a la memoria



Tipos de Fragmentación

Fragmentación
Externa

Espacios libres muy pequeños
entre procesos asignados

Fragmentación
Interna

Espacio desperdiciado
dentro de bloques asignados



Universidad
Tecnológica
del Perú



ALGORITMOS DE ASIGNACIÓN

Algoritmos Clásicos de Asignación

1 First Fit (Primer Ajuste)

Principio: Asigna el primer bloque libre que sea suficientemente grande.

Algoritmo:

```
for each free_block in memory_list: if free_block.size >= requested_size: allocate(free_block, requested_size) return success return failure
```

Ventajas: Rápido, simple de implementar

Desventajas: Puede crear fragmentación al inicio



ALGORITMOS DE ASIGNACIÓN

Algoritmos Clásicos de Asignación

2

Best Fit (Mejor Ajuste)

Principio: Encuentra el bloque libre más pequeño que satisfaga la solicitud.

Algoritmo:

```
best_block = null smallest_size = infinity
for each free_block in memory_list:
    if (free_block.size >= requested_size AND free_block.size < smallest_size):
        best_block = free_block
        smallest_size = free_block.size
if best_block != null:
    allocate(best_block, requested_size)
```

Ventajas: Minimiza el desperdicio de memoria

Desventajas: Más lento, puede crear muchos fragmentos pequeños



ALGORITMOS DE ASIGNACIÓN

Algoritmos Clásicos de Asignación

3 Worst Fit (Peor Ajuste)

Principio: Asigna el bloque libre más grande disponible.

Ventajas: Deja fragmentos grandes que pueden ser útiles

Desventajas: Desperdicia memoria, performance similar a Best Fit

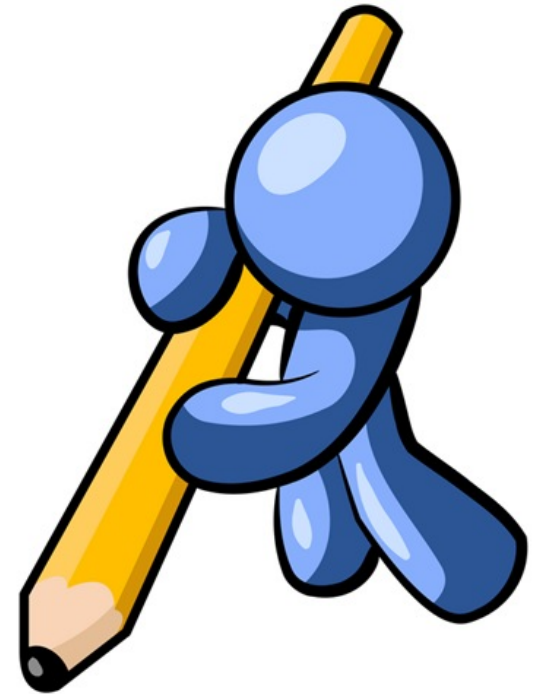
Preguntas

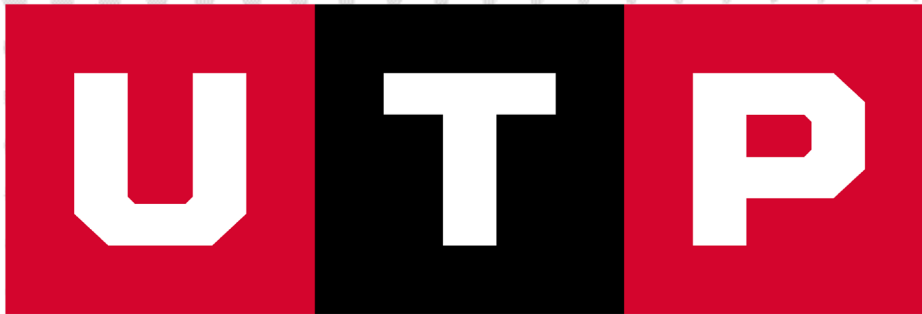
- ¿Tienes alguna duda o comentario?



CIERRE

- ¿Qué aprendiste en esta sesión?
- Te invitamos a compartir tus conclusiones en clase





**Universidad
Tecnológica
del Perú**